Lichen over BitVMX

the dispute, as built and measured

waxwing

Version 0.1, October 2026

a companion to *Lichen*, version 0.2 [1]

Abstract

Lichen [1] lets a user withdraw a hub's layer-2 tokens as bitcoin from a two-party contract with the hub, and settles a refused withdrawal on Bitcoin by checking a Groth16 proof of the withdrawal statement. The main paper treats the on-chain check as a gadget with a narrow interface. This paper describes the gadget that is built: BitVMX's interactive search [5] over the execution of a fixed RISC-V program, the Groth16 verifier, played as a chain of pre-signed transactions between the user and the hub. We give the transaction graph leaf by leaf, the windows, the fees and the payout, and the costs measured on regtest and on Mutinynet with the current statement: a false claim disproved is 100 kvB in 23 transactions, a lie about a memory read 200 kvB in 25, an exit against a vanished hub 23 kvB in 5, and a contract pre-signs 141 transactions. From those figures we derive the reserves a contract needs and the number of contracts a hub can hold before congesting the chain becomes a profitable attack.

Contents

1 Scope **2**

2 The disputed program **3**
2.1 One program for every withdrawal . . . 3
2.2 The input words . . . 4

3 Building blocks **4**
3.1 Values Script can read: Winternitz signatures . . . 4
3.2 Pre-signed chains . . . 4
3.3 BitVMX's search . . . 5
3.4 Reads and the authenticated jump list . . . 5

4 The transaction graph **6**
4.1 The contract output . . . 6
4.2 The escalation and the inputs . . . 7
4.3 The search . . . 7
4.4 The last step . . . 8
4.5 The read and jump challenges . . . 9

4.6 Off-chain play . . . 9

5 Outcomes and the payout **9**

6 Fees **10**

7 Costs **11**
7.1 Transactions . . . 11
7.2 Time . . . 12
7.3 Pre-signing . . . 12

8 Reserves **12**

9 Mass exit **13**
9.1 By bytes . . . 13
9.2 Congestion . . . 13
9.3 What would improve it, within BitVMX . . . 14

10 Alice's liveness and the tower **14**

11 Scenarios **15**

12 Beside the bridges **15**

13 Open questions **16**

14 Implementation **16**

1. Scope

The main paper [1] describes the contract between a user, Alice, and a hub; the layer 2 and its withdrawal statement; anchoring and finality; and the economics of reserves and mass exit, all in terms that do not depend on how a disputed proof is checked on Bitcoin. It asks of the on-chain gadget only this:

> given a proof whose public input binds the program and its journal $b \,\|\, c \,\|\, D$, adjudicate a two-party claim on Bitcoin: the contract's fixed terms (c, D, the program) checked, b readable by Script for a variable payout, the loser paying.

and summarises any gadget by a handful of numbers: the bytes and transactions of an exit against an absent hub and of a dispute played to the end, each side's heaviest move within one window, the dispute's duration, the pre-signed set and the setup per contract.

This paper fills in those numbers for BitVMX, as built in the proof of concept [3, 4], and explains where they come from. Everything upstream of the Groth16 proof (the guests, RISC Zero's STARKs and recursion, the wrap) is unchanged by the choice of gadget and is covered in the main paper and the pipeline note [2]; we start where the proof becomes a claim about a program run. Table 1 collects the results.

parameter	BitVMX, as built
exit against a vanished hub	23.1 kvB, 5 transactions (Mutinynet)
a true claim refused and played to the end	about 120 kvB, 23 transactions (the sum of measured parts; 122.9 kvB measured on regtest with the older 43-word input)
a false claim disproved	100.4 kvB, 23 transactions (Mutinynet)
the worst case: a lie about a read, disproved	200.5 kvB, 25 transactions at 66 hops (Mutinynet); about 212 kvB at the 86-hop maximum (est.)
Alice's heaviest move due within one window	her proof of the last step, 25.2 kvB
the hub's heaviest move due within one window	its path, `combine_8`, about 93 kvB (81.1 kvB measured at 7 chunks)
times Alice must act in a full dispute	13
duration against a staller	about $10\,w_H + w_P$ blocks (hostile hub), $11\,w_A$ (hostile user)
pre-signed per contract	141 transactions, 169 inputs
the links' fee floors (the reserve's floor)	197,356 sats
setup beyond pre-signing	none: no garbling, no trusted setup of its own

Table 1: The gadget's parameters, for the main paper's model. Sizes are virtual bytes as mined, including fee inputs where a transaction takes one; Section 7 gives the parts and their sources.

2. The disputed program

2.1. One program for every withdrawal

Alice's withdrawal proof is a Groth16 proof (BN254) that RISC Zero's recursion verified a succinct receipt of the withdrawal guest, with journal $b \,\|\, c \,\|\, D$ [2]. The claim taken to Bitcoin is about the run of a fixed program on that proof:

> FairgateLabs' Groth16 verifier for BitVMX [6], compiled to RISC-V, run on Alice's 204-byte input, halts with exit code 0.

The verifier's ELF is FairgateLabs' as shipped, pinned by its SHA-256 (`2b96e192...`); the emulator and the scripts are BitVMX-CPU with two bug fixes and the authenticated jump list of Section 3.4 added, in a fork [7]. The verifier has RISC Zero's verifying key, control root and BN254 control id compiled in, so it accepts exactly the receipts of RISC Zero's 3.0 release family. It decompresses the proof's three points, recomputes RISC Zero's claim digest from the image id and the journal, combines the verifying key's points by the public inputs and checks the pairing equation, halting with 0 if it holds. That takes about 478 million RISC-V steps (477,943,091 on a reference input; the count varies by some hundreds of thousands with the input), about a minute in BitVMX's emulator on a laptop, two with the trace hashed.

2.2. The input words

The input is 51 four-byte words. Alice signs each with its own one-time key (Section 3.1) when she takes her claim on chain, so the contract can read some of them and check others against its terms:

words	content	published in	used by the contract
0	the journal's length in words (10)	`escalate`	checked (`journal_len`)
1–8	the withdrawal guest's image id	`escalate`	checked (`image_1`–`image_8`)
9–40	the proof's three points, compressed	`inputs`	by the search only
41	b	`escalate`	read by the payout
42	c, the contract's memo	`escalate`	checked (`memo`)
43–50	D, the contract's identifier	`inputs`	checked (`deployment_1`–`deployment_8`)

Each check is a leaf of the dispute's first output, spendable by the hub alone and at once: it re-verifies Alice's signature on the word and succeeds if the signed value differs from the contract's, taking the whole contract. So a proof of another program (the image id), for another contract (the memo) or against another deployment, history or finality threshold (D) loses before any search. The image-id check is essential: the verifier would accept a valid proof of *any* program. One image id suffices because the withdrawal guest has the chain guest's image id built in. Words are split between two transactions only because of Script's stack (Section 4.2).

3. Building blocks

3.1. Values Script can read: Winternitz signatures

Script checks signatures only over the spending transaction, never over a message of the signer's choosing, so a value chosen during a dispute (an input word, a hash of the trace, a choice) is carried by a Winternitz one-time signature, which Script verifies by hashing alone and which leaves the signed digits on the stack for the rule that reads them [9, 5]. Digits are base 16: a key for n bytes signs $2n$ message digits and a checksum, so a four-byte word has 10 digits, a 20-byte hash 43, a one-byte choice 4. Each digit costs two witness items, a 20-byte chain value and the digit. Keys are derived from each party's seed per contract, exchanged when the contract opens, and used once: signing two values under one key is *equivocation*, which a leaf can prove by showing both signatures, giving the output to the other side.

Script's stack holds at most 1,000 items, the witness counted, so a single input can carry, and a single rule can re-read, only a few hundred signed bytes. Every rule therefore re-reveals only the fields it reads; a move is split into one key per field; and large posts (the input words, the hub's path) are split across transactions or inputs.

3.2. Pre-signed chains

Every step of the dispute is a 2-of-2 transaction between Alice and the hub, signed by both when the contract opens. The values a step carries travel in its witness, which its transaction id does not cover, so every step's id is fixed in advance and the next step can be signed against

it. Pre-signing binds a transaction to the outputs it spends, so a pre-signed chain can be shared only by paths that reach it through the same transaction; here there is one, `escalate`, and the whole dispute is pre-signed once, linear in the search's length. What cannot be pre-signed is a spend whose outputs depend on the moment (a disprove, a check, a verdict, a sweep): these are *runtime* spends of leaves that need only the spender's key and its proof of a lie, and they pay the spender's own wallet.

3.3. BitVMX's search

BitVMX [5] settles a claim about a RISC-V program's run. The prover (Alice) claims that the program, on her signed input, halts after n steps; each step's effect is hashed into a chain $h(1), h(2), \ldots$, and she commits to the last hash. If the verifier (the hub) disagrees, they search. In each round the prover posts the hashes at $k-1$ evenly spaced steps of the interval in dispute, and the verifier names the segment where it first disagrees. With $k = 8$ and at most 2^{29} steps, ten rounds suffice: rounds 1–9 post seven hashes and choose among eight segments (3 bits each), round 10 posts three and chooses among four (2 bits), and the hub's ten choices, concatenated, are the step t they agree on, with disagreement on $h(t+1)$. The prover then posts her record of step $t+1$: the instruction, the values it read (and the step that last wrote each), what it wrote, and the hashes on either side. Either her record proves, by BitVMX's verification script for its class of instruction, that step t's hash leads to step $t+1$'s, or the record disagrees with the program, its input or the search, and the hub disproves it.

3.4. Reads and the authenticated jump list

A step's hash covers what the step writes, not what it reads; a read's value is whatever was last written to its address. So Alice could claim a false value for a read, naming a plausible step as its last write, and execute the instruction faithfully on it: her chain would diverge there, the search would find the step, and her proof of it would pass. BitVMX catches this with a second search for the read's last write, as long again as the first. Here the trace is hashed instead as an *authenticated jump list*, after S. D. Lerner [8]:

$$h(i) = \mathrm{BLAKE3}\big(h(i-1) \,\|\, h(\mathit{jump}(i)) \,\|\, \mathit{write}_i \,\|\, i\big)_{[0,20)},$$

where $\mathit{jump}(i)$ is an earlier step fixed by the number i alone, so that any step reaches any earlier one in at most 86 hops (about 52 on average at 2^{29} steps), each to the previous step or to the long jump. A genuine hash commits to its step number, so its back-links are the genuine hashes of the steps it names. To show a false read, the hub posts the hashes along a path from the agreed $h(t)$ back to the last write, and the write's preimage; its verdict wins if the write contradicts Alice's read (another value at the step she names, or a later write to her address). Alice, holding the true history, may challenge any hop she finds false, and the hub must then reveal that step's preimage, which must hash to the posted hash and link to the next, so a forged path loses at its first false hop. The same path settles a second lie the jump list makes possible: Alice's record commits to the hash of her step's long jump, and a false one would let a valid write prove on a false chain; the hub's path to $\mathit{jump}(t+1)$ shows the true hash. Where BitVMX needs a second search, this needs three or four more transactions (Section 4.5).

4. The transaction graph

Figure 1 shows the graph. Its outputs are *carried* values: each link of the chain moves the whole contract, less a fee floor (Section 6), into the next output, so whichever leaf finally spends an output settles the whole contract.

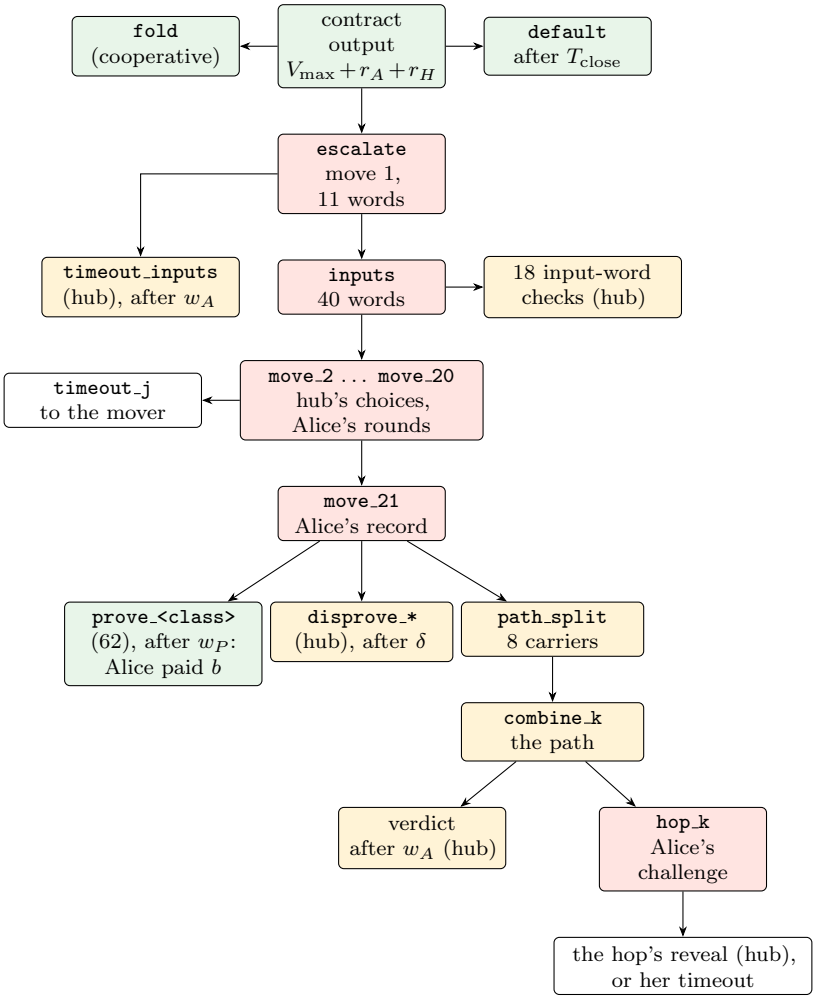


Figure 1: The dispute's graph, simplified. Red: Alice's links and the chain of moves; yellow: the hub's moves and spends; green: settlements. The links are pre-signed. Every link's output also has a pre-signed timeout against the party due to move, and equivocation leaves for every one-time key used so far.

4.1. The contract output

The contract output holds $V_{\max} + r_A + r_H$ and funds no fees. Every taproot tree in the graph has an unspendable (NUMS) internal key, so every spend is a script path; leaves are placed by a Huffman tree of equal weights. The contract output has three leaves:

- `fold`: the 2-of-2, signed when the fold is agreed, not in advance; Alice receives $b + r_A$, the hub the rest less a fixed fee. (A MuSig2 key path would make this about 140 vB instead of

193.)

- `default`: after T_{close} (`CLTV`), pre-signed, returning r_A to Alice and the rest to the hub.
- `escalate`: after the contract's `to_self_delay` (`CSV`), the 2-of-2 and Alice's signatures on move 1 and on her claim words.

The proof of concept builds contracts as standalone outputs; the same three leaves sit in a channel commitment beside the revocation leaf in LN-GAP's channel code [4], where every channel update re-signs the set for both commitments.

4.2. The escalation and the inputs

`escalate` spends the contract into the *claimed output*. Its witness carries move 1 (the claim, 28 bytes: the last step and its hash; and round 1's seven hashes) and 11 claim words: the journal's length, the image id, b and the memo. That is 944 stack items, close to the limit, so the other 40 words (the proof and D) follow in `inputs`, which spends the claimed output into the first output of the search. The claimed output has one more leaf, `timeout_inputs`, paying the hub if `inputs` has not come within Alice's window w_A: the hub must have her whole input before it can run the verifier and move.

The search's first output carries the 18 input-word checks of Section 2, besides the leaves every output has. The hub reads the words from both links and checks them before it moves; a wallet refuses to escalate a claim that fails one.

4.3. The search

After `inputs`, the search alternates the hub's choices (moves 2, 4, ..., 20) and Alice's rounds of hashes (3, 5, ..., 19), ending with her record (move 21). Move j is the link `move_`j, spending output $j-1$ into output j; each link has two outputs, the carried value and an *anchor* for its poster's fees. Every output $j < 21$ has:

- `post`: `1 CSV`, the 2-of-2 and the next mover's signatures on its move's fields: the pre-signed `move_`$j{+}1$;
- `timeout_`j: the 2-of-2 after the next mover's window, pre-signed, paying the contract to the mover of j, since its opponent failed to answer;
- the opponent's rules on move j, as runtime `disprove` leaves after a delay δ: for a choice, that it is out of range;
- `equiv_d`: one leaf for each field of each move so far, by which the other side shows two signatures under one key and takes the output (Section 4.6).

A round of Alice's is seven 20-byte hashes, 301 signed digits and 606 witness items: about 7.5 kvB. A choice of the hub's is one byte, about 0.34 kvB. The `1 CSV` on `post` means the opponent's answer can only follow a link's confirmation, so nothing but the poster's own fee child can spend an unconfirmed link (Section 6).

Windows by obligation. A move's size, not its mover, decides how long it may need to get into a block when blocks are full (Section 9), so the windows are set by what is owed: Alice has

w_A for each of her moves and for `inputs`; the hub has w_H for each choice, a few hundred bytes; and w_P for its heavy moves at the end, the path and the disproves' opportunity before Alice may prove. The hub offers its windows with each contract; a wallet refuses a w_A shorter than the network's minimum. As configured:

network	w_A	w_H	w_P	δ
regtest	14	10	16	6
Mutinynet (30-second blocks)	80	20	80	40
signet	12	6	12	6
testnet4	72	18	72	36

Alice's window and the path's must exceed δ, so that every runtime disprove opens before the timeout it races.

4.4. The last step

Alice's final move signs five fields: her 64-byte record of step $t+1$; $h(t)$ and $h(t+1)$; J, the hash of the step's long jump; and $t+1$ itself. The record holds the step's two reads (address, value and the step that last wrote each), its program counter and opcode, its write (address, value, next program counter) and the agreed step t. It is about 6.8 kvB. Its output has the largest tree of the graph, about 2,200 leaves by our count:

- one `prove_<class>` per class of instruction in the program, 62 for the Groth16 verifier: after w_P, pre-signed, paying Alice's payout. Its script guards the class, runs BitVMX's verification script for it on the record, and checks the jump-list step hash BLAKE3($h(t) \,\|\, J \,\|\, write \,\|\, t+1) = h(t+1)$. It re-reveals all five fields and is Alice's heaviest move: 25.2 kvB as mined, its fee input included.
- the hub's runtime disproves, after δ, each paying it the output: `step` (the record's t is not the hub's choices), `initial`, `halt_hash`, `halt_exit` (the claimed last step is not an exit with code 0: 5.1 kvB), `entry_point`, `program_counter`, `addresses_sections`, `succ`, `future_read_1` and `_2`, the equivocation re-sign challenges for every round and index (132 leaves), `uninitialized_1` and `_2`, `initialized_`k for each data chunk, `opcode_`k for each of the program's 1,773 code chunks (the opcode at the record's pc), and `input_`j for each input word (a read of an input word never written must equal Alice's signed word, re-revealed from `escalate` or `inputs`);
- `path`: `1 CSV` and the 2-of-2, the hub's way into the read and jump challenges below;
- `timeout_21`: after $w_P + w_A$, the hub's, if Alice never proves;
- the `equiv_d` leaves, 82 by the final output.

The hub decides by running BitVMX's challenge selection on the record: a mismatch with the program or the input picks the disprove; a read it contests, or a long-jump hash that differs from its own, picks the path; otherwise the step proves and an honest hub stops.

4.5. The read and jump challenges

A path of up to 86 hashes, 1,720 signed bytes, does not fit one input's stack, so it is posted in chunks, one per input of one transaction. From the final output, the hub's `path_split` makes eight *carriers*, the first holding the value and the rest dust, each with a `chunk` leaf verifying one chunk's one-time signatures; `combine_`k spends the first k carriers, chunk c in input c. The first chunk holds the hops' count, their types (a nibble each: previous step or long jump), the path's end and 8 hashes; each other chunk holds 11. Both transactions are pre-signed, a combine for each k. The path's output then has:

- `hop`: `1 CSV`, the 2-of-2 and Alice's one-byte choice of a hop to challenge: the pre-signed `hop_`k;
- the hub's verdicts, `jump`, `read_1` and `read_2`, after w_A, each with the end step's 57-byte preimage: the end is $jump(t+1)$ and its hash differs from J, or the end is the read's last write and contradicts Alice's read;
- `timeout_combine_`k: after $w_P + w_A$, Alice's, if no verdict comes.

After `hop_`k, the hub has δ to reveal the challenged hop's preimage, which must hash to its posted hash and link to the next by the hop's type; if it cannot, Alice's `timeout_hop_`k pays her after w_P. A split never combined pays Alice from the first carrier after w_P. A read dispute adds three or four transactions to the search: the split, a combine and the verdict, and Alice's challenge with its answer if she makes one.

4.6. Off-chain play

LN-GAP lets the parties play the search off chain by exchanging signed moves, and lets either take it on chain by *replaying* the moves so far from the signatures it holds; the `equiv_d` leaves keep a replay faithful, since altering one of one's own moves means signing a second value under its key. lichen does not play off chain: an honest hub that checks Alice's proof natively, in milliseconds, has nothing to gain from a search it will lose, and a hub that refuses is better met on chain at once. The leaves are built and tested in LN-GAP [4] and cost only tree depth here. What lichen does replay is its own state: after a restart, each side re-derives its moves, deterministic from its seed and the chain, and checks them against the links already mined.

5. Outcomes and the payout

A dispute decides only Alice's withdrawal: b if her claim stands, nothing if it fails; the rest of $V_{\max}$ goes to the hub, and the loser's reserve to the winner.

When Alice wins (a proof of the last step; any timeout against the hub; a path split and never combined, a verdict that never comes, a hop not revealed) the terminal transaction pays a pre-signed *payout*: one output of 2^i sats for each bit i of a 32-bit amount from bit 9 up to the contract's size K (so b is rounded down to a multiple of 512 sats, and $V_{\max} = 2^K - 1$), an output for the reserves, and the remainder to the hub. Script cannot compute an amount, but it can read Alice's signature on word 41, so each bit output has three leaves, as for numeric DLCs [13]:

leaf	spent by	condition
`alice_bit_`i	Alice, after δ	her signature on b has bit i set
`hub_bit_`i	the hub	the same signature has bit i clear
`equiv_b_`i	the hub	two signatures on different values of b

Since b is under a one-time key, the bits Alice can take are those of the one value she signed, and they sum to b; her leaves wait δ so that an equivocation can be shown first. The reserve output holds $r_A + r_H$ less the links' floors, with Alice's leaf after δ and an `equiv_b` leaf of its own. A bit's spend is about 320–330 vB, and each party sweeps its outputs in one transaction; the hub finds its own by Alice's signature, already on chain in `escalate`.

When the hub wins (a check, a disprove, a verdict, a hop's reveal, or a timeout against Alice) the hub takes the output whole, both reserves included, as a runtime spend to its wallet, or by a pre-signed timeout paying one output.

The variable payout is not BitVMX's. Any gadget needs a way for Script to read b, and the bit outputs need only the one-time signature on it, so this section carries over unchanged to other gadgets; the main paper describes it as part of the contract.

6. Fees

Links (`escalate`, `inputs`, the moves, the split, the combines, the hops) must keep their transaction ids, since the next link was signed against them, so nothing can be added to them. Each is an ordinary version-2 transaction that pays a *floor* of 1 sat/vB of its own out of the carried value, enough to relay alone, and carries a 330-sat anchor that only its poster can spend; when the feerate is higher, the poster adds a child spending the anchor and a coin of its own. Because the opponent's next move waits a block after the link confirms (`1 CSV`), only the poster's child can sit on an unconfirmed link: the challenge and response happen on chain, not in the mempool, and there is nothing for pinning to work on. So a link may be as large as standard relay allows (100 kvB), where TRUC's rules would cap it at 10 kvB; the escalation is 11.6 kvB and a path's combine up to 93. A floor is computed from an upper bound of its link's size, every digit at full length and the leaf at its tree's greatest depth, so the floors are a property of the statement, not of the keys. The deepest dispute, a worst-case read, spends 197,356 sats of floors and anchors, about half of it in the combine.

Terminals (the timeouts, the proofs, the default) are spent only by runtime leaves, so their ids may change. Both parties sign them `ALL|ANYONECANPAY` at no fee, committing to their input and all their outputs, and the broadcaster adds one coin, sized exactly, with no change; the feerate is chosen at broadcast. Alice keeps a fee budget for this, by default 30,000 sats where fees are low and 150,000 elsewhere, and a wallet refuses a contract whose heaviest terminal at the minimum feerate, `prove_<class>` at about 26,800 sats, the budget could not pay.

Runtime spends (checks, disproves, verdicts, sweeps) are built when needed and pay what the moment requires from the value they take.

7. Costs

7.1. Transactions

Table 2 gives each transaction's size as mined. Unless marked, figures are from the Mutinynet trials of October 2026 [3] (contracts 1001–1006), with the current 51-word input, at 1 sat/vB and so without fee children.

transaction	size
`fold` (cooperative close)	193 vB
`escalate`: move 1 and 11 words (470 digits, 944 items)	11.6 kvB
`inputs`: 40 words (400 digits)	9.9 kvB
the hub's choice	0.33–0.35 kvB
Alice's round of seven hashes; her round 10 (three)	7.5 kvB; 3.4 kvB
her record (move 21)	6.8 kvB
`prove_<class>`, her proof of the last step	25.2 kvB (an upper bound of 26.2 over all classes)
`disprove_halt_exit`	5.1 kvB
`path_split`; `combine_`k; `disprove_read_2`	0.6 kvB; 11.6 kvB a chunk (81.1 kvB for 7, a 66-hop path); 23.6 kvB
a timeout, with its fee input	0.59–0.60 kvB
a bit output's spend	0.32–0.33 kvB
an input-word check	0.41 kvB (regtest)

Table 2: Sizes as mined on Mutinynet unless marked. The escalation varies by a few bytes with its leaf's depth in the tree.

Whole disputes, as played:

case	size	txs	source
the hub vanished: escalate, inputs, a split for the fee coin, the hub's timeout, the payout's sweep	23.1 kvB	5	Mutinynet, contract 1001
the hub returns mid-dispute and concedes on its native check	24.0 kvB	5	Mutinynet, 1002
a false claim (an exit-code lie), disproved by `halt_exit`	100.4 kvB	23	Mutinynet, 1003
a lie about a read, disproved through a 66-hop path	200.5 kvB	25	Mutinynet, 1004
a true claim refused and played to the end	about 120 kvB	23	the sum of the parts above; 122.9 kvB measured on regtest with the 43-word input, fee children included

A true claim played through is 22 links and its terminal: escalation and inputs 21.5 kvB, Alice's

rounds 63.6, the hub's choices 3.4, her record 6.8 and her proof 25.2. Exchanging the proof for the hub's `halt_exit` gives 100.4 kvB, the false claim exactly as measured. At the jump list's 86-hop maximum a read's combine is about 93 kvB and the read dispute about 212 kvB.

7.2. Time

item	measured
the withdrawal proof (STARK), the current guest	389,020 cycles: 74 s on an Apple M4 Pro; 359 s on a 4-vCPU x86 server
the Groth16 wrap	257 s on the same server (x86 only)
checking the proof natively	1.5 ms; a returning hub conceded 2.4 s after it started (contract 1002)
the verifier, emulated	about a minute; two with the trace hashed
both sides' executions over a search	about 12 minutes
a dispute played on regtest (51 words)	a true claim proved in 730 s, a false one disproved in 545 s
a hostile hub played to the payout on Mutinynet, Alice's side run by a tower	the record 15 minutes after the escalation; the payout 75 minutes after the tower started (30-second blocks)

On mainnet the windows set the time. A hub that stalls on every move stretches a dispute to about $10\,w_H + w_P$ blocks, and a block a move besides; a user who stalls, to about $11\,w_A$, her own money waiting. A vanished hub's users wait only the hub's first window, w_H, after their input. With $w_A = 144$, $w_H = 12$ and $w_P = 288$, a hostile hub can stretch a dispute to about 430 blocks, three days, and a vanished hub's users wait two hours after their input.

7.3. Pre-signing

A contract pre-signs 141 transactions with 169 inputs: `default`; `escalate` and `timeout_inputs`; the 21 links from `inputs` to `move_21` and their 21 timeouts; the 62 proofs; and the tail (`path_split` and its timeout, and for each k from 1 to 8, `combine_`k, `hop_`k and their timeouts). The hub builds the set and signs; Alice rebuilds it herself, verifies the hub's 169 signatures and signs; the hub verifies hers before it broadcasts the funding. A mock statement for tests needs 47.

8. Reserves

The main paper argues that each side's reserve must at least cover the other side's fees in a dispute it loses, and that the hub's binds, since it must make Alice whole for a refused withdrawal however small her balance. Here that floor is concrete. The links' floors alone, fixed when the contract is signed, reach 197,356 sats in the deepest dispute; lichen's hub sets r_H to the larger of its share of the cap and that figure. At a prevailing feerate f, what the reserve must cover is roughly Alice's side of a dispute, about 117 kvB (escalation and inputs 21.5, her rounds 63.6, her record 6.8, her proof 25.2), times f: 117,000 sats at 1 sat/vB, 234,000 at 2, 1.2 million at 10. On top of that the hub locks the cap's headroom $V_{\max} - D$, so a user holding 50,000 sats costs the hub about as much locked capital as one holding two million; under this gadget, the contract suits balances from about half a million sats, and comfortably from a few million.

9. Mass exit

9.1. By bytes

exit	per user	per block	per day; per month, at a quarter of each block
the hub present: a fold and its sweep	0.3 kvB	3,400	120,000; 3.7M
the hub gone	23.1 kvB	43	1,600; 47,000
the hub refuses a true claim to the end	120 kvB	8	300; 9,000
the hub disproves a false claim	100.4 kvB	10	360; 10,800
the hub disproves a read lie	200–212 kvB	5	170; 5,100

A day is 144 blocks, a month 4,320, a block 1 MvB. With the hub gone, each exit has until its contract's T_{close}, so staggered expiries spread the load.

9.2. Congestion

When one side is hostile, a different limit binds. To make an honest party miss a move, the attacker must outbid it in every block of that move's window of w blocks. That costs about the same whether one defender is shut out or many, so it pays only when the moves due in one window could not all fit in it anyway: $Ns > w \cdot 1\,\text{MvB}$, for N contracts whose heaviest move due in a window is s. This is Flood & Loot [12] in lichen's form. We take the attacker's best case, every contract's heaviest move due in the same window: hostile users choose when to escalate, and a hostile hub can delay each of its replies by up to w_H to bunch them.

	a hostile hub, against honest users	hostile users (read lies), against an honest hub
the defender's heaviest move in one window, s	Alice's proof of the last step, 25.2 kvB	the hub's `combine_8`, about 93 kvB
contracts safe at $w = 12$ (two hours)	480	130
at $w = 144$ (a day)	5,700	1,550
at $w = 1{,}008$ (a week)	40,000	10,800
the window needed for 3,000 contracts	76 blocks (13 hours)	280 blocks (2 days)
for 60,000	1,510 blocks (10 days)	5,580 blocks (39 days)

The escalation and `inputs` (21.5 kvB) are not counted: they run against T_{close}, not a window, and only `inputs` has a window, Alice's own, opened by her own escalation. Each window holds only its own moves, so only the two heavy ones grow with N: Alice's proof, due in the w_A after w_P, and the hub's path, in w_P. The hub's choices, 0.34 kvB each, keep w_H short, and that is the window a vanished hub's users wait. The second column binds: the liar picks the lie, and a read lie maximises the hub's load in one window.

9.3. What would improve it, within BitVMX

- *The path in its own windows.* Posting each chunk of the path as its own transaction in its own window, only in the read-lie tail, would bring the hub's s down to its read verdict, 23.6 kvB: about 6,000 contracts per day-long window instead of 1,550.
- *A shorter jump list.* Lerner's tighter long-jump function, about $\log_2 n + 3\log_2\log_2 n$ hops (some 45 instead of 86), or a counter moving over a step number's set bits only, shrinks the path, its combine and the floors about in proportion.
- *The inputs on demand.* With the hub gone, Alice could escalate with only b and the checked words; a present hub would post a small pre-signed demand, and only then would she owe the rest. The absent-hub exit would fall from 23 kvB to about 3–4 (est.).
- *Delegated fold signing.* Judging a fold needs only public layer-2 data, so a successor or backup key (even the sequencer's) could sign folds while the hub is down; a rogue backup can only overpay users, the hub's own risk. An outage then costs 0.3 kvB an exit, not 23.
- *A chess clock.* Union [11] gives each party a budget for all its moves together, so a staller stretches the dispute only to the two budgets, not to ten windows; the price is extra pre-signed transactions, published only against a staller.

Smaller rounds would not help Alice's side: her heaviest move is the proof of one step, whatever the search's arity.

10. Alice's liveness and the tower

A dispute runs for hours on Mutinynet and days on mainnet, and Alice must answer each of the hub's choices within w_A. Her moves follow from her trace, which anyone with her input can compute, but a move is also her commitment: hashes from her trace signed with her one-time keys for that move, and which hashes depends on the hub's earlier choices, so the signatures cannot be made in advance. That is what makes delegation under BitVMX harder than in Lightning, where a watchtower holds pre-signed penalties.

As built. lichen's tower runs the wallet's own dispute engine from a *package* Alice exports: the contract's record with the whole pre-signed set; b, as her signature rather than its key, so the tower cannot claim another amount; the seeds of her other input-word keys, her move keys and her hop keys; her fee budget's key; and, optionally, her withdrawal input and chain receipt so that it can prove the withdrawal itself. It reads the chain through two Esplora servers that must agree, plays the whole dispute, read challenges included, to the payout, and stops: Alice sweeps with her wallet, which checks that the payout is a terminal of her contract. The worst a tower can do is make wrong moves, equivocate, stay silent or spend the fee budget, so it is trusted like her own machine. The intended form is her own: a container on a rented server, started when a fold is refused and discarded after, since in a standalone contract only Alice can start a dispute and nothing can be broadcast against her before she does. In the Mutinynet trials such a tower, on a 4-vCPU server, proved the withdrawal and played a hostile hub to the payout in about 75 minutes.

Keyless moves would let anyone post them: Alice would sign a Merkle root over her trace when she escalates, and each later move would be an opening of it. The opening must be checked when posted, or a dishonest poster could publish garbage first; without `OP_CAT` a 29-level path cannot be hashed natively, and emulating the hash in Script would make each move hundreds of times larger. With `OP_CAT` it would be cheap.

11. Scenarios

scenario	outcome under BitVMX
the hub disappears	Alice escalates and posts her inputs; the hub never moves; `timeout_1` pays b and both reserves
the hub refuses a valid claim	the search on chain; Alice proves the last step; b and both reserves
Alice claims a return she never made	the verifier halts with failure; the hub's `halt_exit` takes everything
Alice props up a false claim with a false read, or a false long jump	the hub's path through its history to the read's last write (or to the long jump); unless she shows a hop false, its verdict takes everything
the hub forges a hash in that path	Alice challenges the hop; the hub cannot reveal it; her timeout pays b and both reserves
the hub splits the path and never combines it	the first carrier's timeout pays b and both reserves
Alice escalates and never posts her inputs	`timeout_inputs` pays the hub
Alice probes: a false claim, abandoned when the hub answers	she loses by timeout: everything, her reserve included, to the hub
a proof of another program, for another contract or against another history or work	an input-word check takes the contract before any search
Alice signs two values of b	the hub takes the bit outputs and the reserves with `equiv_b`
either side signs two values under one move key	the other takes the output with `equiv_d`
either side forces the search on chain to cause trouble	the outcome is unchanged; each pays its own moves' fees beyond the floors, and the loser's reserve covers the floors

12. Beside the bridges

BitVMX's search is also the dispute of Union [11], a bridge to Rootstock by BitVMX's own authors, where every pair of a committee of functionaries pre-signs a game for every peg-in, 4-ary, about 270 kvB at worst, with a nested game for a challenge to the claimant's chain of headers. A lichen contract is Union's game with a single pair, 8-ary with a jump list in place of the second search, and with the user as the claimant for her own withdrawal; the main paper compares the custody and trust of the two designs. BitVM2 [10] takes the opposite route: since anyone may challenge, nothing interactive can be pre-signed, and its dispute is a single round in which the operator posts the verifier's intermediate state at once, with scripts of up to 4 MB.

13. Open questions

- *An escalation late in the contract.* `escalate` has no upper time bound and `default` opens at T_{close}, so an escalation broadcast near T_{close} races the default. A wallet should escalate with room to spare; a script bound would remove the race.
- *Pinning of terminals.* Links cannot be pinned, but a terminal signed `ALL|ANYONECANPAY` can be, through any output spendable before confirmation (the hub's share of a payout, the default's outputs). A `1 CSV` on every such output, as on the links and as in Lightning's anchor channels, would close this.
- *Package relay.* Links pay a floor of 1 sat/vB and, when mempools are full, rely on relaying the link with its poster's child.
- *Built but unused leaves.* The hub's `step` and `succ` disproves and Alice's `choice` rule are in the tree and tested as scripts, but neither engine fires them; the `equiv_d` leaves serve an off-chain play lichen does not use.
- *A channel embedding* would re-sign the 141 transactions for both commitments on every channel update.
- *The reductions of Section 9*, none of them built.

14. Implementation

The contract layer is LN-GAP's [4]: the search's schedule, the one-time-signature scripts, the ladder of links and its rules, the jump list and the read tail, and the pre-signed set. lichen [3] supplies the statement (the input words and their checks), the payout's parameters, the hub's and the wallet's dispute engines, the tower, and the deployment on Mutinynet. BitVMX-CPU is used from a fork with the jump list and two fixes [7]. The regtest tests play a withdrawal proved and folded, a true claim refused and proved, a false claim disproved and a read lie refuted through a path, each on the Groth16 verifier; the Mutinynet trials played an absent hub, a hub that returns and concedes, an exit-code lie and a read lie, and, with Alice's side run by a tower, a hostile hub to the payout. The regtest Groth16 tests have not yet been re-run on the guests rebuilt after the latest review.

References

[1] waxwing. Lichen: L2s in channels enforced by Bitcoin. Version 0.2, October 2026. https://github.com/AdamISZ/lichen

[2] waxwing. From a Rust program to one step on Bitcoin: how a withdrawal is proved, wrapped and disputed. October 2026. https://github.com/AdamISZ/ln-gap/blob/master/docs/paper/pipeline.pdf

[3] lichen: the ledger, the sequencer, the hub and the wallet. https://github.com/AdamISZ/lichen

[4] LN-GAP proof-of-concept code. https://github.com/AdamISZ/ln-gap

[5] S. D. Lerner, R. Amela, S. Mishra, M. Jonas and J. Álvarez Cid-Fuentes. BitVMX: a CPU for universal computation on Bitcoin, 2024. https://arxiv.org/abs/2405.06842

[6] FairgateLabs. BitVMX-CPU: the emulator, the n-ary search and the Groth16 verifier for BitVMX. https://github.com/FairgateLabs/BitVMX-CPU

[7] BitVMX-CPU with LN-GAP's changes (two bug fixes, the authenticated jump list), branch `lngap`. https://github.com/AdamISZ/BitVMX-CPU

[8] S. D. Lerner. Latest innovations in BitVMX (trace Merkle tree, authenticated skiplist and jumplist), June 2024. https://bitvmx.org/knowledge/latest-innovations-in-bitvmx

[9] R. Linus. BitVM: compute anything on Bitcoin, 2023. https://bitvm.org/bitvm.pdf

[10] R. Linus Woll, L. Aumayr, Z. Avarikioti, M. Maffei, A. Pelosi, O. S. Thyfronitis Litos, C. Stefo, D. Tse and A. Zamyatin. Bridging Bitcoin to second layers via BitVM2. USENIX Security 2026; ePrint 2025/1158. https://eprint.iacr.org/2025/1158

[11] R. Amela, S. Mishra, S. D. Lerner and J. Álvarez Cid-Fuentes. Union: a trust-minimized bridge for Rootstock, 2025. https://arxiv.org/abs/2501.07435

[12] J. Harris and A. Zohar. Flood & Loot: a systemic attack on the Lightning Network. AFT 2020. https://arxiv.org/abs/2006.08513

[13] DLC specifications: numeric outcome compression. https://github.com/discreetlogcontracts/dlcspecs